12576 - Nisanth Saravanan
.Net Core and Entity Framework Lab Assessment

Collaborative Coding Environment

A console application that enables users to collaboratively write, edit, and share code in real time. It supports multi-user project management, live code editing, and version control, making it ideal for remote teams, coding bootcamps, and peer programming sessions.

User Stories to be Developed

1. User Registration and Project Creation

   As a user, I want to register and create coding projects, so that I can collaborate with others on shared codebases..

2. Real-Time Code Editing

   As a user, I want to edit code files in real time with other collaborators, so that we can work together efficiently on the same project.

3. Upload and Manage Code Files

   As a user, I want to upload, organize, and manage code files within a project, so that our team can maintain a structured and accessible codebase.

Task:

- Perform W3H analysis on the given Case Study.
- Use the same Case Study and Create a Two Console application to implement both Code-First Approach and Database-First Approach.
- Connect to MySql server and create database and tables as needed and implement table relationships [one to one or one to many or many to many] as required.
- Finally, perform all required database operations using the Entity Framework on the given entities as per use case.

Documents to be submitted in PDF file format.

1.W3H doc.
2.Output Screen doc.
3. Coding Screens
4. DB Screens with Data Persistence and Relationships.
5. https://github.com/benhar-velankanni/RelevantzRIX2024.git use your respective Branches to upload your Project to GitHub.

W3H Analysis

Project Title: Collaborative Coding Environment	
What?	**How?**
1. What are the modules needed? A: The required modules are: • User Registration and Login. • User Dashboard. • Project Creator. • Code Editor. **2. What are the features of User Registration and Login?** A: User Registration and Login: • Create User. • Login User. **3. What are the features of User Dashboard?** A: User Dashboard: • Show Profile details. • Change Password. • Show Project details. **4. What are the features of Project Creator?** A: Project Creator: • Create Projects. • Delete Projects. • Update Projects. **5. What are the features of Code Editor?** A: Code Editor: • Create new Code. • Edit Code.	**1. User Registration and Login:** **a) Create User:** **Method 1: Use simple forms.** Method 2: Use GoogleOAuth only. Method 3: Use other third party accounts. **b) Login User:** **Method 1: Using simple forms.** Method 2: Use GoogleOAuth only. Method 3: Use other third party accounts. **2. User Dashboard:** **a) Show Profile details:** **Method 1: Fetch details from the Database.** Method 2: Fetch the user's google info. Method 3: Simply generate a random username for a user. **b) Change Password:** **Method 1: Using nodemailer to reset their password.** Method 2: Use a modal. Method 3: Use OPT based system to reset password. **c) Show Project details:** **Method 1: Fetch details from database.** Method 2: Fetch details from a locally setup logger. Method 3: Fetch details from cache. **3. Project Creator:** **a) Create Projects:** **Method 1: Use simple forms.**

• Delete Code.	Method 2: Use modular forms. Method 3: Create from a set of pre-made templates. **b) Delete Projects:** **Method 1: Delete by Project ID.** Method 2: Delete by Project Name. Method 3: Delete by random sequence. **c) Update Projects:** **Method 1: Update details by ID.** Method 2: Update details by Name. Method 3: Update details using modular forms. **4. Code Editor:** **a) Create new Code:** **Method 1: Use simple text editor.** Method 2: Re-route to a specific editor. Method 3: Only allow file uploads. **b) Edit Code:** **Method 1: Edit by Code ID.** Method 2: Edit by Code File Name. Method 3: Refuse Editing after uploading. **c) Delete Code:** **Method 1: Delete by Code ID.** Method 2: Delete by Code File Name Name. Method 3: Delete by date of creation.
1. User Registration and Login: **a) Create User:** **Method 1: Use simple forms.** A: Proves to be an efficient choice, without complicating the flow of the program. **b) Login User:** **Method 1: Using simple forms.** A: Proves to be an efficient choice, without complicating the flow of the program.	**1. User Registration and Login:** **a) Create User:** Method 2: Use GoogleOAuth only. Method 3: Use other third party accounts. A: As per the current context, the methods are inefficient or simply unnecessary. **b) Login User:** Method 2: Use GoogleOAuth only. Method 3: Use other third party accounts. A: As per the current context, the methods are inefficient or simply

2. User Dashboard: **a) Show Profile details:** **Method 1: Fetch details from the Database.** A: Proves to be an efficient choice, without complicating the flow of the program. **b) Change Password:** **Method 1: Using nodemailer to reset their password.** A: Proves to be an efficient choice, without complicating the flow of the program. **c) Show Project details:** **Method 1: Fetch details from database.** A: Proves to be an efficient choice, without complicating the flow of the program. **3. Project Creator:** **a) Create Projects:** **Method 1: Use simple forms.** A: Proves to be an efficient choice, without complicating the flow of the program. **b) Delete Projects:** **Method 1: Delete by Project ID.** A: Proves to be an efficient choice, without complicating the flow of the program. **c) Update Projects:** **Method 1: Update details by ID.** A: Proves to be an efficient choice, without complicating the flow of the program. **4. Code Editor:**	unnecessary. **2. User Dashboard:** **a) Show Profile details:** Method 2: Fetch the user's google info. Method 3: Simply generate a random username for a user. A: As per the current context, the methods are inefficient or simply unnecessary. **b) Change Password:** Method 2: Use a modal. Method 3: Use OPT based system to reset password. A: As per the current context, the methods are inefficient or simply unnecessary. **c) Show Project details:** Method 2: Fetch details from a locally setup logger. Method 3: Fetch details from cache. A: As per the current context, the methods are inefficient or simply unnecessary. **3. Project Creator:** **a) Create Projects:** Method 2: Use modular forms. Method 3: Create from a set of pre-made templates. A: As per the current context, the methods are inefficient or simply unnecessary. **b) Delete Projects:** Method 2: Delete by Project Name. Method 3: Delete by random sequence. A: As per the current context, the methods are inefficient or simply unnecessary.

a) Create new Code: **Method 1: Use simple text editor.** A: Proves to be an efficient choice, without complicating the flow of the program. **b) Edit Code:** **Method 1: Edit by Code ID.** A: Proves to be an efficient choice, without complicating the flow of the program. **c) Delete Code:** **Method 1: Delete by Code ID.** A: Proves to be an efficient choice, without complicating the flow of the program.	**c) Update Projects:** Method 2: Update details by Name. Method 3: Update details using modular forms. A: As per the current context, the methods are inefficient or simply unnecessary. **4. Code Editor:** **a) Create new Code:** Method 2: Re-route to a specific editor. Method 3: Only allow file uploads. A: As per the current context, the methods are inefficient or simply unnecessary. **b) Edit Code:** Method 2: Edit by Code File Name. Method 3: Refuse Editing after uploading. A: As per the current context, the methods are inefficient or simply unnecessary. **c) Delete Code:** Method 2: Delete by Code File Name Name. Method 3: Delete by date of creation. A: As per the current context, the methods are inefficient or simply unnecessary.
Why?	**Why Not?**

Code First Approach and DB First Approach

Migrations

```
nisanth@RLP1865:~/DotNetAssessment/CodingEnvironment_CF$ dotnet ef migrations add InitialCreate
dotnet ef database update
Build started...
Build succeeded.
Done. To undo this action, use 'ef migrations remove'
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this
takes too long.
Applying migration '20250630114459_InitialCreate'.
Done.
```

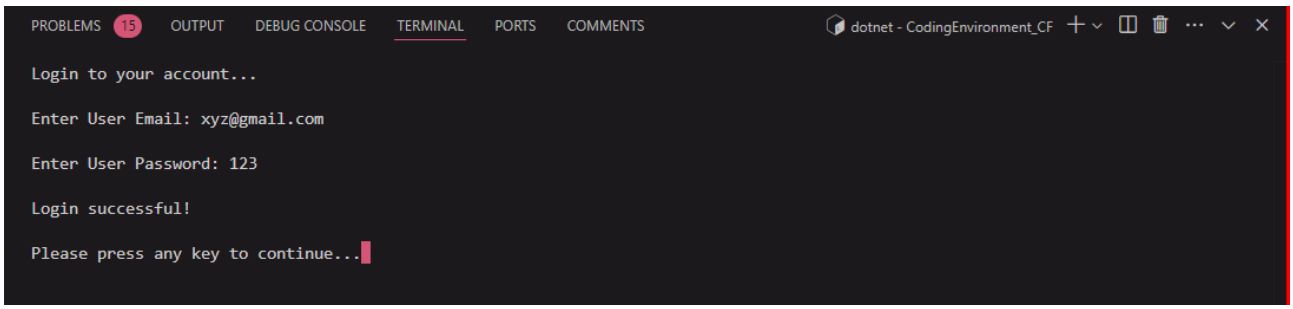
Output

```
PROBLEMS 15  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS        dotnet - CodingEnvironment_CF

Welcome to Collabo - Your Coding Environment...

Select an option:

1. Login
2. Register
0. Exit

Enter your choice:
```

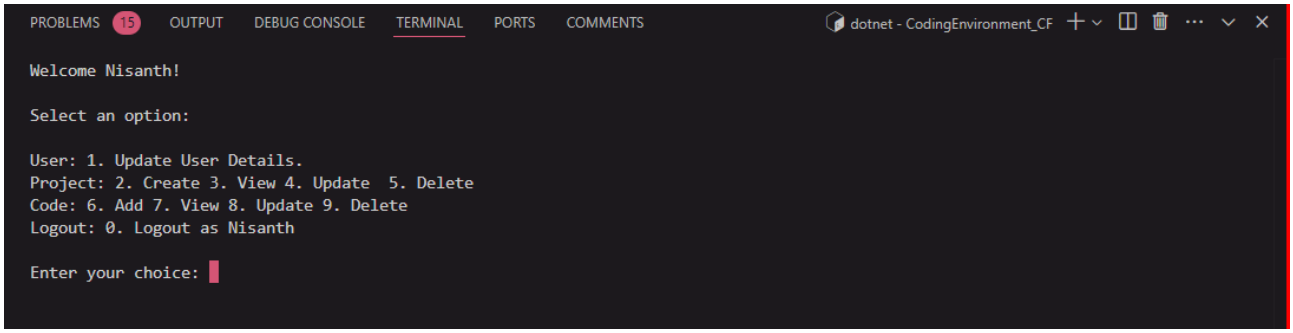
Register

```
PROBLEMS 15  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  COMMENTS        dotnet - CodingEnvironment_CF

Adding a new User...

Enter User Name: Nisanth

Enter User Email: dasff
User email is not valid.

Enter User Email: xyz@gmail.com

Enter User Password: 123

Adding User to database...
User added successfully!

Please press any key to continue...
```

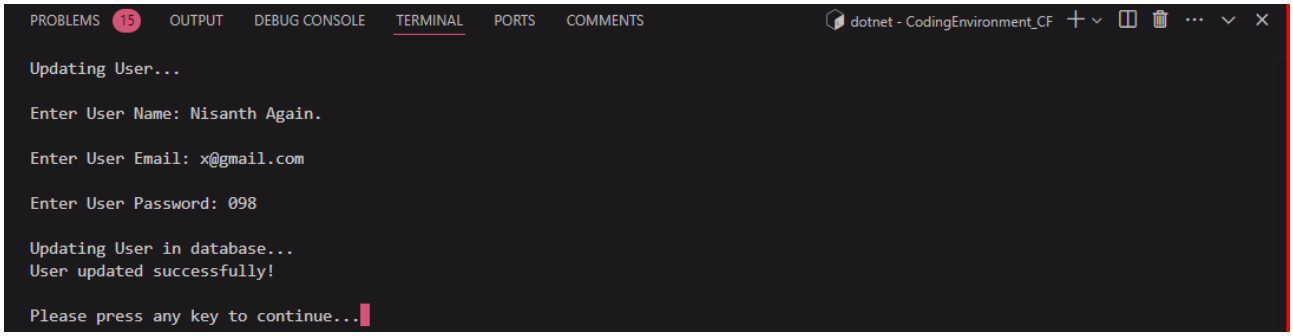
Login



Dash Board

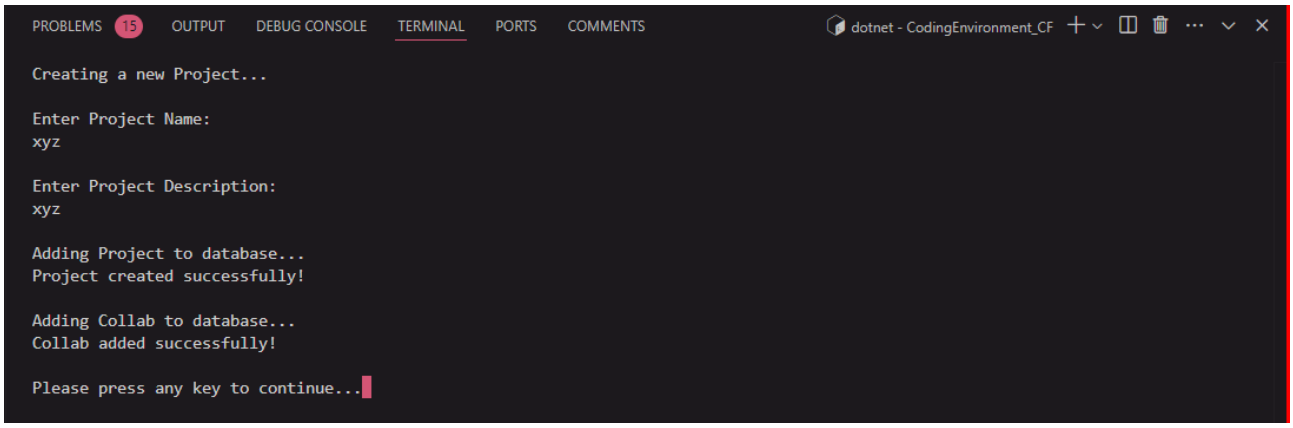


Update Profile

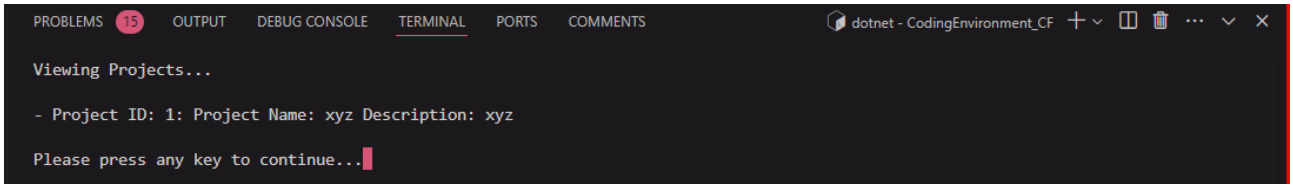


Projects

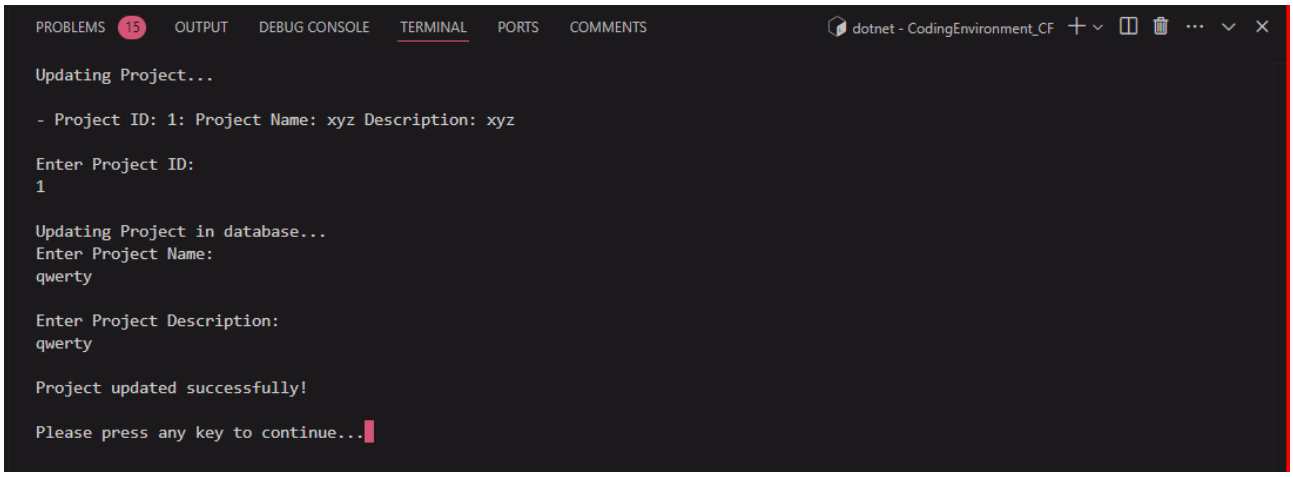
Create



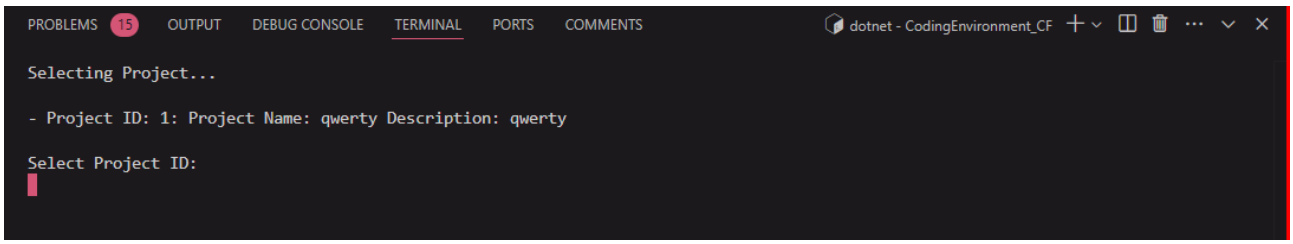
View



Update

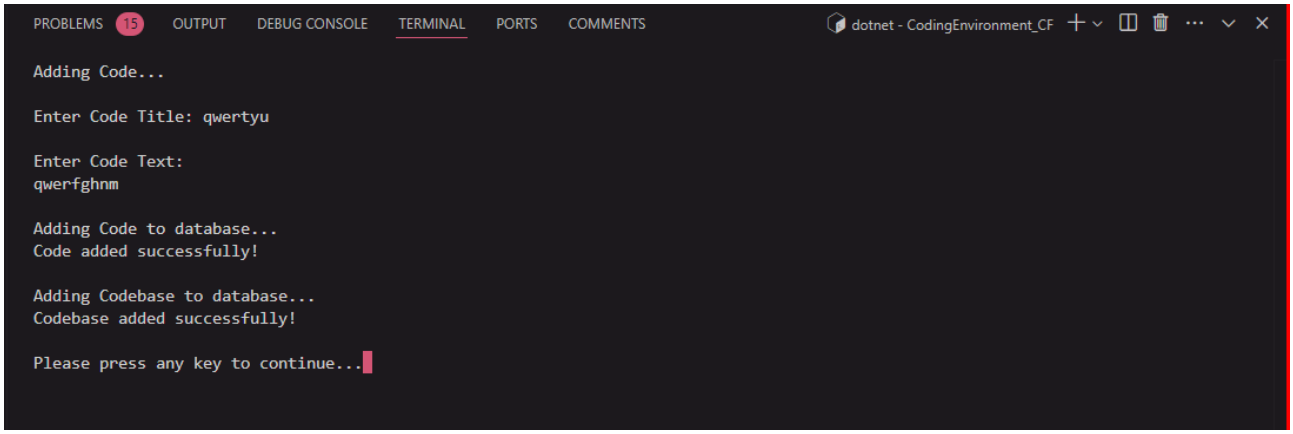


Project Selector

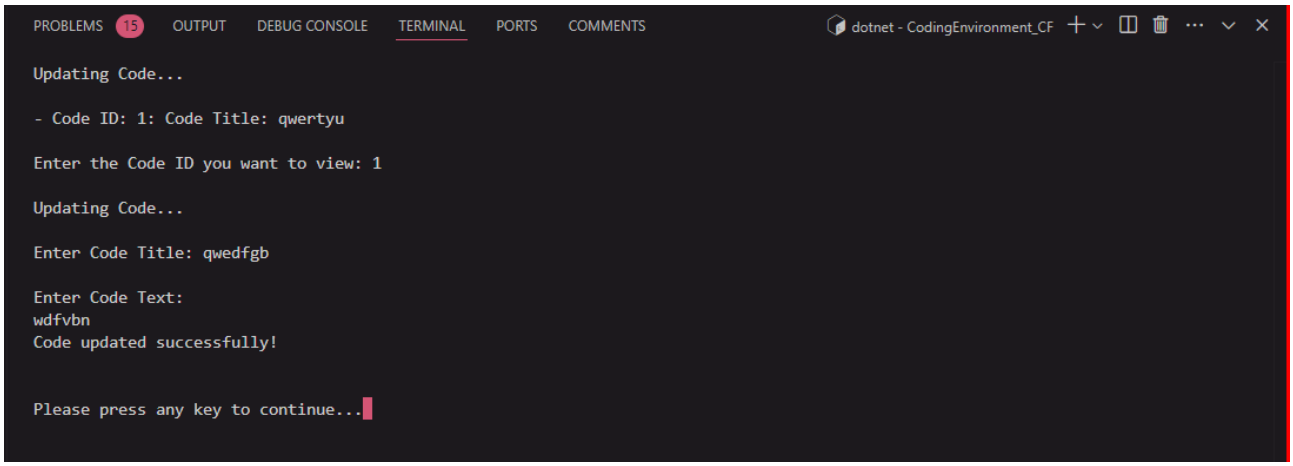


Codes

Create

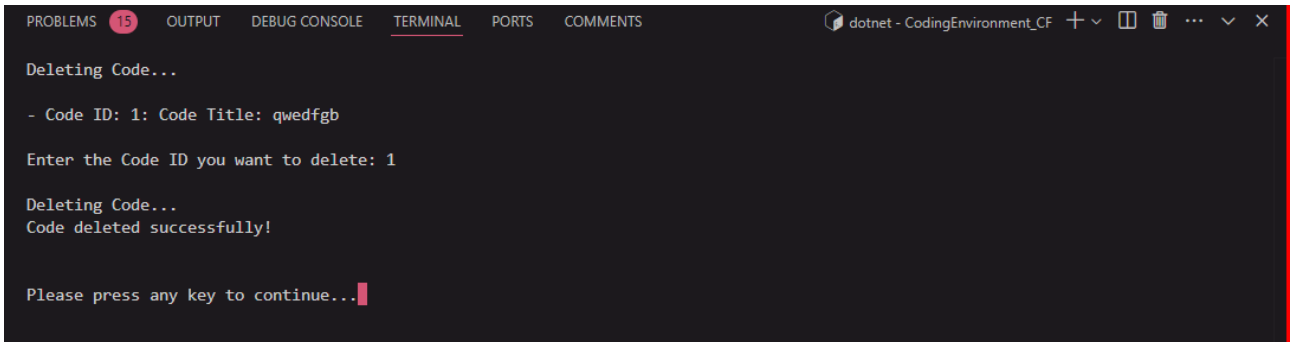


Update

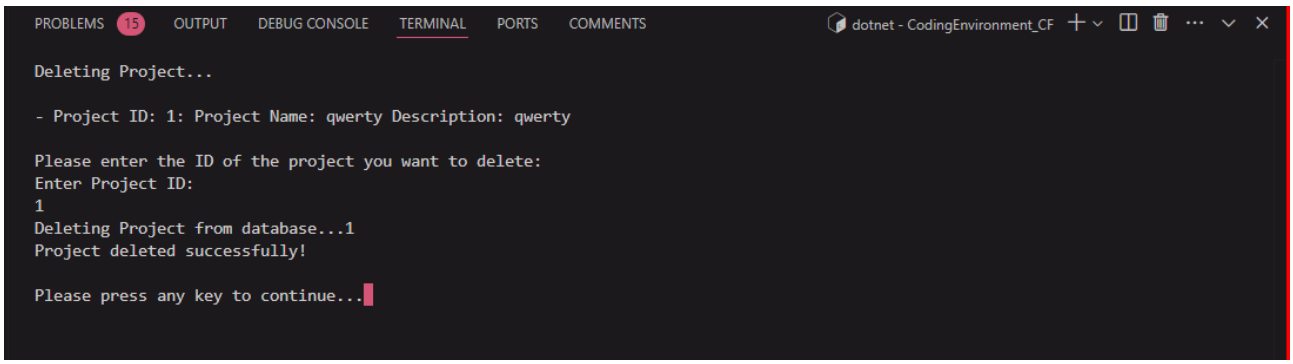


Deletions

Codes

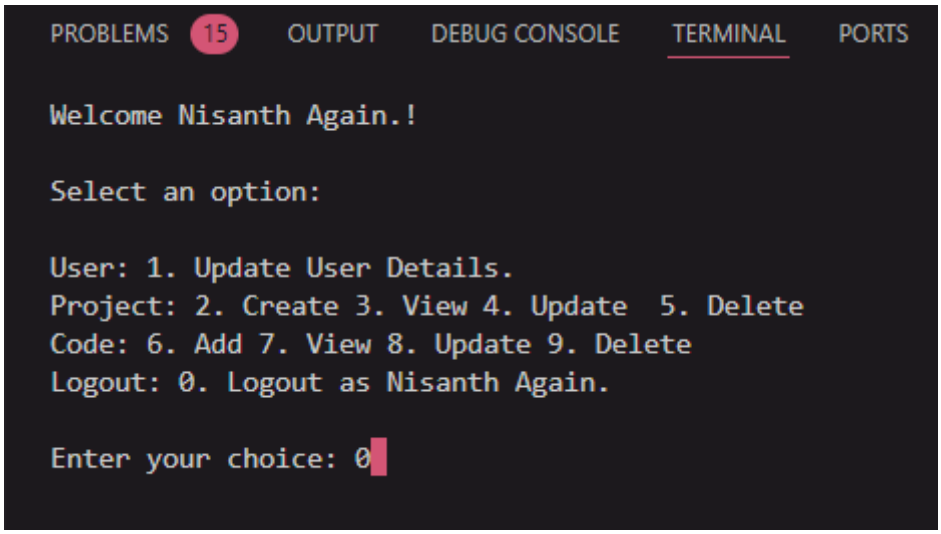


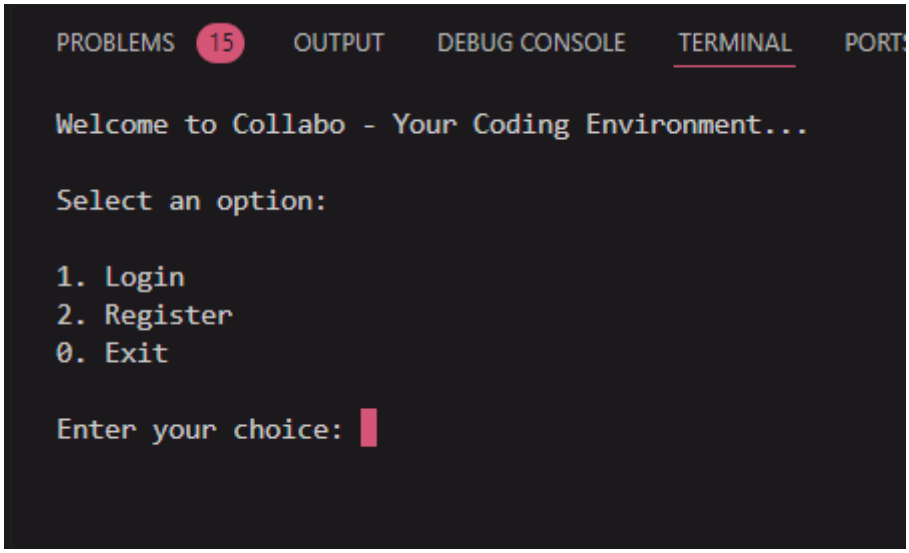
Projects



Exiting:

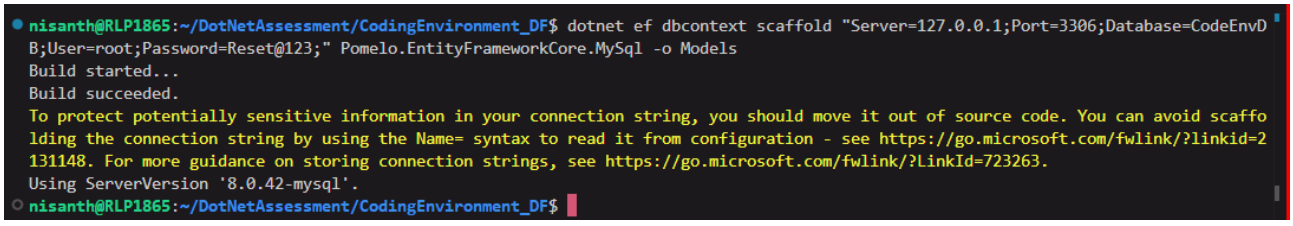
Exiting from a logged in state brings you to home:





All inputs have dedicated null checking and regex (only for email) validation.

Scaffolding



Code: From Code First Approach

Data

```
namespace CodingEnvironment.Data;

public class CodeEnvDBContext : DbContext
{
  protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
  {
    var connectionString =
"Server=127.0.0.1;Port=3306;Database=CodeEnvDB;User=root;Password=Reset@123;";
    optionsBuilder.UseMySql(connectionString,
ServerVersion.AutoDetect(connectionString));
  }

  protected override void OnModelCreating(ModelBuilder modelBuilder)
  {
    //Keys
    modelBuilder.Entity<User>()
      .HasIndex(u => u.UserId)
      .IsUnique();

    modelBuilder.Entity<Project>()
      .HasIndex(p => p.ProjectId)
      .IsUnique();

    modelBuilder.Entity<Code>()
      .HasIndex(c => c.CodeId)
      .IsUnique();

    modelBuilder.Entity<Collab>()
      .HasIndex(c => c.CollabId)
      .IsUnique();

    modelBuilder.Entity<CodeBase>()
      .HasIndex(c => c.CodeBaseId)
      .IsUnique();

    //RelationShips
    modelBuilder.Entity<Collab>()
      .HasOne(c => c.User)
      .WithMany(u => u.Collabs)
      .HasForeignKey(c => c.UserId);

    modelBuilder.Entity<Collab>()
      .HasOne(c => c.Project)
      .WithMany(p => p.Collabs)
      .HasForeignKey(c => c.ProjectId);
```

```
    modelBuilder.Entity<CodeBase>()
      .HasOne(c => c.Project)
      .WithMany(p => p.Codes)
      .HasForeignKey(c => c.ProjectId);

    modelBuilder.Entity<CodeBase>()
      .HasOne(c => c.Code)
      .WithMany(c => c.Codes)
      .HasForeignKey(c => c.CodeId);
  }

  public DbSet<User> Users { get; set; }
  public DbSet<Project> Projects { get; set; }
  public DbSet<Code> Codes { get; set; }
  public DbSet<Collab> Collabs { get; set; }
  public DbSet<CodeBase> CodeBases { get; set; }
}
```

Models

Code

```
namespace CodingEnvironment.Model;

public class Code
{
  [Key]
  public int CodeId { get; set; }
  public string? CodeTitle { get; set; }
  public string? CodeText { get; set; }
  public DateTime CreatedDate { get; set; }
  public DateTime LastModifiedDate { get; set; }

  [ForeignKey("Project")]
  public int ProjectId { get; set; }
  public Project? Project { get; set; }

  [ForeignKey("User")]
  public int AuthorId { get; set; }
  public User? Author { get; set; }

  [ForeignKey("User")]
  public int LastModifierId { get; set; }
  public User? LastModifier { get; set; }


  public IEnumerable<CodeBase>? Codes { get; set; }
}
```

Code Base

```
namespace CodingEnvironment.Model;

public class CodeBase
{
  [Key]
  public int CodeBaseId { get; set; }

  [ForeignKey("Project")]
  public int ProjectId { get; set; }
  public Project? Project { get; set; }

  [ForeignKey("Code")]
  public int CodeId { get; set; }
  public Code? Code { get; set; }

  public IEnumerable<CodeBase>? Codes { get; set; }
}
```

Collab

```
namespace CodingEnvironment.Model;

public class Collab
{
  [Key]
  public int CollabId { get; set; }

  [ForeignKey("User")]
  public int UserId { get; set; }
  public User? User { get; set; }

  [ForeignKey("Project")]
  public int ProjectId { get; set; }
  public Project? Project { get; set; }
}
```

Project

```
namespace CodingEnvironment.Model;

public class Project
{
  [Key]
  public int ProjectId { get; set; }
  public string? ProjectName { get; set; }
  public string? ProjectDescription { get; set; }

  [ForeignKey("User")]
  public int AdminId { get; set; }
  public User? Admin { get; set; }

  public IEnumerable<Collab>? Collabs { get; set; }

  public IEnumerable<CodeBase>? Codes { get; set; }
}
```

User

```
namespace CodingEnvironment.Model;

public class User
{
  [Key]
  public int UserId { get; set; }
  public string? UserName { get; set; }
  public string? UserEmail { get; set; }
  public string? UserPassword { get; set; }

  public IEnumerable<Collab>? Collabs { get; set; }
}
```

Repositories

User

```
namespace CodingEnvironment.Repo;

public class UserRepo
{
  public static void RegisterUser()
  {
    Console.Clear();
    Console.WriteLine("Adding a new User...");
    Console.WriteLine();

    var userName = GetUserName();
    Console.WriteLine();
    var userEmail = GetUserEmail();
    Console.WriteLine();
    var userPassword = GetUserPassword();
    Console.WriteLine();

    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var user = new User
        {
          UserName = userName,
          UserEmail = userEmail,
          UserPassword = userPassword
        };

        Console.WriteLine("Adding User to database...");
        context.Users.Add(user);
        context.SaveChanges();
        Console.WriteLine("User added successfully!\n");

        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine($"\nAn error occurred: {ex.Message}...\n");

      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }
```

```
  public static List<User> LoginUser()
  {
    Console.Clear();
    Console.WriteLine("Login to your account...");
    Console.WriteLine();

    var userEmail = GetUserEmail();
    Console.WriteLine();
    var userPassword = GetUserPassword();
    Console.WriteLine();

    try
    {
      using (var context = new CodeEnvDBContext()) {
        var users = context.Users.Where(u => u.UserEmail == userEmail &&
u.UserPassword == userPassword).ToList();
        if (users.Count > 0)
        {
          Console.WriteLine("Login successful!\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return users;
        }
        else
        {
          Console.WriteLine("Login failed. Please try again.\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return null;
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine($"\nAn error occurred: {ex.Message}...\n");
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
      return null;
    }
  }

  public static void UpdateUser(int UserIdToUpdate)
  {
    Console.Clear();
    Console.WriteLine("Updating User...");
    Console.WriteLine();

    var userName = GetUserName();
    Console.WriteLine();
    var userEmail = GetUserEmail();
```

```
  Console.WriteLine();
  var userPassword = GetUserPassword();
  Console.WriteLine();

  try
  {
    using (var context = new CodeEnvDBContext())
    {
      var user = context.Users.Find(UserIdToUpdate);
      user.UserName = userName;
      user.UserEmail = userEmail;
      user.UserPassword = userPassword;

      Console.WriteLine("Updating User in database...");
      context.SaveChanges();
      Console.WriteLine("User updated successfully!\n");

      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }
  catch (Exception ex)
  {
    Console.WriteLine($"\nAn error occurred: {ex.Message}...\n");
    Console.Write("Please press any key to continue...");
    Console.ReadKey();
  }
}

private static string GetUserName()
{
  while (true)
  {
    Console.Write("Enter User Name: ");
    var userName = Console.ReadLine();
    if (userName == null)
    {
      Console.WriteLine("User name cannot be empty.\n");
      continue;
    }
    else
    {
      return userName;
    }
  }
}

private static string GetUserEmail()
{
  while (true)
  {
```

```
        Console.Write("Enter User Email: ");
        var userEmail = Console.ReadLine();

        Regex regex = new Regex(@"^([\w\.\-]+)@([\w\-]+)((\.(\w){2,3})+)$");

        if (userEmail == null)
        {
          Console.WriteLine("User email cannot be empty.\n");
          continue;
        }
        else if (!regex.IsMatch(userEmail))
        {
          Console.WriteLine("User email is not valid.\n");
          continue;
        }
        else
        {
          return userEmail;
        }
      }
    }

    private static string GetUserPassword()
    {
      while (true)
      {
        Console.Write("Enter User Password: ");
        var userPassword = Console.ReadLine();
        if (userPassword == null)
        {
          Console.WriteLine("User password cannot be empty.\n");
          continue;
        }
        else
        {
          return userPassword;
        }
      }
    }
}
```

Project

```
using System.Reflection.Metadata.Ecma335;

namespace CodingEnvironment.Repo;

public class ProjectRepo
{
  public static void CreateProject(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Creating a new Project...");
    Console.WriteLine();

    var projectName = GetProjectName();
    Console.WriteLine();
    var projectDescription = GetProjectDescription();
    Console.WriteLine();
    var adminId = UserId;

    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var project = new Project
        {
          ProjectName = projectName,
          ProjectDescription = projectDescription,
          AdminId = adminId
        };

        Console.WriteLine("Adding Project to database...");
        context.Projects.Add(project);
        context.SaveChanges();
        Console.WriteLine("Project created successfully!\n");

        var collab = new Collab
        {
          UserId = UserId,
          ProjectId = project.ProjectId
        };

        Console.WriteLine("Adding Collab to database...");
        context.Collabs.Add(collab);
        context.SaveChanges();
        Console.WriteLine("Collab added successfully!\n");

        Console.Write("Please press any key to continue...");
```

```
        Console.ReadKey();
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }

  public static void ViewProjects(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Viewing Projects...");
    Console.WriteLine();

    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();

        if (collabs.Any())
        {
          foreach (var collab in collabs)
          {
            if (collab.UserId == UserId)
            {
              var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
              foreach (var project in projects)
              {
                Console.WriteLine($"- Project ID: {project.ProjectId}: Project
Name: {project.ProjectName} Description: {project.ProjectDescription}");
              }
            }
          }

          Console.WriteLine();
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
        }
        else
        {
          Console.WriteLine("You are not a member of any project. Please create a
project first.");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return;
```

```
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }

  public static void UpdateProject(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Updating Project...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();

      if (collabs.Any())
      {
        foreach (var collab in collabs)
        {
          if (collab.UserId == UserId)
          {
            var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
            foreach (var project in projects)
            {
              Console.WriteLine($"- Project ID: {project.ProjectId}: Project Name:
{project.ProjectName} Description: {project.ProjectDescription}");
            }
          }
        }
      }
      else
      {
        Console.WriteLine("You are not a member of any project. Please create a
project first.");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
        return;
      }
    }

    Console.WriteLine();
    var projectId = GetProjectId();
    Console.WriteLine();
```

```
    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var project = context.Projects.Find(projectId);
        if (project == null)
        {
          Console.WriteLine("Project not found.\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return;
        }
        else
        {
          Console.WriteLine("Updating Project in database...");
          var projectName = GetProjectName();
          Console.WriteLine();
          var projectDescription = GetProjectDescription();
          Console.WriteLine();

          project.ProjectName = projectName;
          project.ProjectDescription = projectDescription;

          context.SaveChanges();
          Console.WriteLine("Project updated successfully!\n");

          Console.Write("Please press any key to continue...");
          Console.ReadKey();
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }

  public static void DeleteProject(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Deleting Project...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();
```

```
      if (collabs.Any())
      {
        foreach (var collab in collabs)
        {
          if (collab.UserId == UserId)
          {
            var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
            foreach (var project in projects)
            {
              Console.WriteLine($"- Project ID: {project.ProjectId}: Project Name:
{project.ProjectName} Description: {project.ProjectDescription}");
            }
          }
        }
      }
      else
      {
        Console.WriteLine("You are not a member of any project. Please create a
project first.");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
        return;
      }
    }

    Console.WriteLine();
    Console.WriteLine("Please enter the ID of the project you want to delete: ");
    var projectId = GetProjectId();

    try
    {
      Console.WriteLine("Deleting Project from database..." + projectId);
      using (var context = new CodeEnvDBContext())
      {
        var project = context.Projects.Find(projectId);
        if (project == null)
        {
          Console.WriteLine("Project not found.\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return;
        }
        else
        {
          context.Projects.Remove(project);
          context.SaveChanges();
          var collabs = context.Collabs.Where(c => c.ProjectId ==
projectId).ToList();
          foreach (var collab in collabs)
          {
```

```
            context.Collabs.Remove(collab);
            context.SaveChanges();
          }
          Console.WriteLine("Project deleted successfully!\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
}


private static int? GetProjectId()
{
  while (true)
  {
    Console.WriteLine("Enter Project ID: ");
    int? projectId = int.Parse(Console.ReadLine());
    if (projectId == null)
    {
      Console.WriteLine("Project ID cannot be empty.\n");
      continue;
    }
    else
    {
      return projectId;
    }
  }
}

private static string GetProjectName()
{
  while (true)
  {
    Console.WriteLine("Enter Project Name: ");
    var projectName = Console.ReadLine();
    if (projectName == null)
    {
      Console.WriteLine("Project name cannot be empty.\n");
      continue;
    }
    else
    {
      return projectName;
    }
```

```
    }
  }

  private static string GetProjectDescription()
  {
    while (true)
    {
      Console.WriteLine("Enter Project Description: ");
      var projectDescription = Console.ReadLine();
      if (projectDescription == null)
      {
        Console.WriteLine("Project description cannot be empty.\n");
        continue;
      }
      else
      {
        return projectDescription;
      }
    }
  }
}
```

Code

```
namespace CodingEnvironment.Repo;

public class CodeRepo
{
  public static int? SelectProj(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Selecting Project...");
    Console.WriteLine();
    using (var context = new CodeEnvDBContext())
    {
      var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();

      if (collabs.Any())
      {
        foreach (var collab in collabs)
        {
          if (collab.UserId == UserId)
          {
            var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
            foreach (var project in projects)
            {
              Console.WriteLine($"- Project ID: {project.ProjectId}: Project Name:
{project.ProjectName} Description: {project.ProjectDescription}");
            }
          }
        }
      }
      else
      {
        Console.WriteLine("You are not a member of any project. Please create a
project first.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
        return null;
      }
    }

    Console.WriteLine();
    Console.WriteLine("Select Project ID: ");
    int? projectId = int.Parse(Console.ReadLine());
    return projectId;
  }
```

```
public static void AddCode(int UserId)
{
  int? selectedProjectId = SelectProj(UserId);

  if (selectedProjectId == null)
  {
    return;
  }

  Console.Clear();
  Console.WriteLine("Adding Code...");
  Console.WriteLine();

  var codeTitle = GetCodeTitle();
  Console.WriteLine();
  var codeText = GetCodeText();
  Console.WriteLine();

  try
  {
    using (var context = new CodeEnvDBContext())
    {
      var code = new Code
      {
        CodeTitle = codeTitle,
        CodeText = codeText,
        ProjectId = selectedProjectId.Value,
        CreatedDate = DateTime.Now,
        LastModifiedDate = DateTime.Now,
        AuthorId = UserId,
        LastModifierId = UserId,
      };

      Console.WriteLine("Adding Code to database...");
      context.Codes.Add(code);
      context.SaveChanges();
      Console.WriteLine("Code added successfully!\n");

      var codebase = new CodeBase
      {
        ProjectId = selectedProjectId.Value,
        CodeId = code.CodeId
      };
      Console.WriteLine("Adding Codebase to database...");
      context.CodeBases.Add(codebase);
      context.SaveChanges();
      Console.WriteLine("Codebase added successfully!\n");

      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
```

```
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }



  public static void ViewCode(int UserId)
  {
    int? selectedProjectId = SelectProj(UserId);

    if (selectedProjectId == null)
    {
      return;
    }

    Console.Clear();
    Console.WriteLine("Viewing Code...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var codebase = context.CodeBases.Where(c => c.ProjectId ==
selectedProjectId).ToList();
      if (codebase.Any())
      {
        foreach (var codeBase in codebase)
        {
          var codeView = context.Codes.Where(c => c.CodeId ==
codeBase.CodeId).FirstOrDefault();
          Console.WriteLine($"- Code ID: {codeView.CodeId}: Code Title:
{codeView.CodeTitle}");
        }

        Console.WriteLine();
        Console.Write("Enter the Code ID you want to view: ");
        var codeId = int.Parse(Console.ReadLine());

        var code = context.Codes.Where(c => c.CodeId == codeId).FirstOrDefault();
        Console.WriteLine($"- Code Title: {code.CodeTitle}");
        Console.WriteLine($"- Code Text: \n{code.CodeText}");

        Console.WriteLine();
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
      else
```

```
      {
        Console.WriteLine("You have no codes to view.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
  }

  public static void UpdateCode(int UserId)
  {
    int? selectedProjectId = SelectProj(UserId);

    if (selectedProjectId == null)
    {
      return;
    }

    Console.Clear();
    Console.WriteLine("Updating Code...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var codebase = context.CodeBases.Where(c => c.ProjectId ==
selectedProjectId).ToList();
      if (codebase.Any())
      {
        foreach (var codeBase in codebase)
        {
          var codeView = context.Codes.Where(c => c.CodeId ==
codeBase.CodeId).FirstOrDefault();
          Console.WriteLine($"- Code ID: {codeView.CodeId}: Code Title:
{codeView.CodeTitle}");
        }

        Console.WriteLine();
        Console.Write("Enter the Code ID you want to view: ");
        var codeId = int.Parse(Console.ReadLine());
        foreach (var code in codebase)
        {
          if (code.CodeId == codeId)
          {
            Console.WriteLine("\nUpdating Code...");
            Console.WriteLine();
            var newCodeTitle = GetCodeTitle();
            Console.WriteLine();
            var newCodeText = GetCodeText();
            code.Code.CodeTitle = newCodeTitle;
            code.Code.CodeText = newCodeText;
            context.SaveChanges();
            Console.WriteLine("Code updated successfully!\n");
```

```
            break;
          }
          else
          {
            Console.WriteLine("Invalid Code ID. Please try again.");
          }
        }

        Console.WriteLine();
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }else
      {
        Console.WriteLine("You have no codes to update.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
  }

  public static void DeleteCode(int UserId)
  {
    int? selectedProjectId = SelectProj(UserId);

    if (selectedProjectId == null)
    {
      return;
    }

    Console.Clear();
    Console.WriteLine("Deleting Code...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var codebase = context.CodeBases.Where(c => c.ProjectId ==
selectedProjectId).ToList();
      if (codebase.Any())
      {
        foreach (var codeBase in codebase)
        {
          var codeView = context.Codes.Where(c => c.CodeId ==
codeBase.CodeId).FirstOrDefault();
          Console.WriteLine($"- Code ID: {codeView.CodeId}: Code Title:
{codeView.CodeTitle}");
        }

        Console.WriteLine();
        Console.Write("Enter the Code ID you want to delete: ");
        var codeId = int.Parse(Console.ReadLine());
        foreach (var code in codebase)
```

```
        {
          if (code.CodeId == codeId)
          {
            Console.WriteLine("\nDeleting Code...");
            context.Codes.Remove(code.Code);

            var codebaseToDelete = context.CodeBases.Where(c => c.CodeId ==
codeId).FirstOrDefault();
            context.CodeBases.Remove(codebaseToDelete);
            context.SaveChanges();
            Console.WriteLine("Code deleted successfully!\n");
          }
          else
          {
            Console.WriteLine("Invalid Code ID. Please try again.");
          }
        }

        Console.WriteLine();
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
      else
      {
        Console.WriteLine("You have no codes to delete.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
  }

  private static string GetCodeTitle()
  {
    while (true)
    {
      Console.Write("Enter Code Title: ");
      var codeTitle = Console.ReadLine();
      if (codeTitle == null)
      {
        Console.WriteLine("Code title cannot be empty.\n");
        continue;
      }
      else
      {
        return codeTitle;
      }
    }
  }

  private static string GetCodeText()
  {
```

```
    while (true)
    {
      Console.WriteLine("Enter Code Text: ");
      var codeText = Console.ReadLine();
      if (codeText == null)
      {
        Console.WriteLine("Code text cannot be empty.\n");
        continue;
      }
      else
      {
        return codeText;
      }
    }
  }


}
```

Global Using

```
global using Microsoft.EntityFrameworkCore;

global using System.Text.RegularExpressions;

global using System.ComponentModel.DataAnnotations;
global using System.ComponentModel.DataAnnotations.Schema;

global using CodingEnvironment.Model;
global using CodingEnvironment.Data;
global using CodingEnvironment.Repo;
```

Program.cs

```
public class Program
{
  public static void Main(string[] args)
  {
    while (true)
    {
      Console.Clear();
      Console.WriteLine("Welcome to Collabo - Your Coding Environment...");

      Console.WriteLine("\nSelect an option:");
      Console.WriteLine();
      Console.WriteLine("1. Login \n2. Register \n0. Exit");

      Console.Write("\nEnter your choice: ");
      var choice = Console.ReadLine();

      switch (choice)
      {
        case "1":
          {
            List<User> LoggedIn = UserRepo.LoginUser();
            if (LoggedIn.Any())
            {
              Logged(LoggedIn);
            }
            else
            {
              Console.WriteLine("Login failed. Please try again.\n");
              Console.Write("Press any key to continue...");
              Console.ReadKey();
              continue;
            }
            break;
          }
        case "2":
          {
            UserRepo.RegisterUser();
            break;
          }
        case "0":
          {
            Environment.Exit(0);
            break;
          }
        default:
          {
            Console.WriteLine("Invalid choice. Please try again.");
            break;
```

```
        }
    }
  }
}

public static void Logged(List<User> users)
{
  while (true)
  {
    Console.Clear();
    Console.WriteLine("Welcome " + users[0].UserName + "!");
    Console.WriteLine();

    Console.WriteLine("Select an option:");
    Console.WriteLine();
    Console.WriteLine("User: 1. Update User Details.");
    Console.WriteLine("Project: 2. Create 3. View 4. Update  5. Delete");
    Console.WriteLine("Code: 6. Add 7. View 8. Update 9. Delete");
    Console.WriteLine($"Logout: 0. Logout as {users[0].UserName}");

    Console.Write("\nEnter your choice: ");
    var choice = Console.ReadLine();

    switch (choice)
    {
      case "1":
      {
        UserRepo.UpdateUser(users[0].UserId);
        break;
      }
      case "2":
      {
        ProjectRepo.CreateProject(users[0].UserId);
        break;
      }
      case "3":
      {
        ProjectRepo.ViewProjects(users[0].UserId);
        break;
      }
      case "4":
      {
        ProjectRepo.UpdateProject(users[0].UserId);
        break;
      }
      case "5":
        {
          ProjectRepo.DeleteProject(users[0].UserId);
          break;
        }
        case "6":
```

```
            {
              CodeRepo.AddCode(users[0].UserId);
              break;
            }
            case "7":
            {
              CodeRepo.ViewCode(users[0].UserId);
              break;
            }
            case "8":
            {
              CodeRepo.UpdateCode(users[0].UserId);
              break;
            }
            case "9":
            {
              CodeRepo.DeleteCode(users[0].UserId);
              break;
            }
          case "0":
          {
            return;
          }
          default:
            {
              Console.WriteLine("Invalid choice. Please try again.");
              break;
            }
        }
      }
    }
}
```

Code: From DB First Approach

Data

```
using System;
using System.Collections.Generic;
using Microsoft.EntityFrameworkCore;
using Pomelo.EntityFrameworkCore.MySql.Scaffolding.Internal;

namespace CodingEnvironment_DF.Models;

public partial class CodeEnvDBContext : DbContext
{
  public CodeEnvDBContext()
  {
  }

  public CodeEnvDBContext(DbContextOptions<CodeEnvDBContext> options)
    : base(options)
  {
  }

  public virtual DbSet<Code> Codes { get; set; }

  public virtual DbSet<CodeBasis> CodeBases { get; set; }

  public virtual DbSet<Collab> Collabs { get; set; }

  public virtual DbSet<EfmigrationsHistory> EfmigrationsHistories { get; set; }

  public virtual DbSet<Project> Projects { get; set; }

  public virtual DbSet<User> Users { get; set; }

  protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
#warning To protect potentially sensitive information in your connection string,
you should move it out of source code. You can avoid scaffolding the connection
string by using the Name= syntax to read it from configuration - see
https://go.microsoft.com/fwlink/?linkid=2131148. For more guidance on storing
connection strings, see https://go.microsoft.com/fwlink/?LinkId=723263.
    =>
optionsBuilder.UseMySql("server=127.0.0.1;port=3306;database=CodeEnvDB;user=root;pa
ssword=Reset@123", Microsoft.EntityFrameworkCore.ServerVersion.Parse("8.0.42-
mysql"));

  protected override void OnModelCreating(ModelBuilder modelBuilder)
  {
    modelBuilder
      .UseCollation("utf8mb4_0900_ai_ci")
```

```
      .HasCharSet("utf8mb4");

    modelBuilder.Entity<Code>(entity =>
    {
      entity.HasKey(e => e.CodeId).HasName("PRIMARY");

      entity.HasIndex(e => e.AuthorId, "IX_Codes_AuthorId");

      entity.HasIndex(e => e.CodeId, "IX_Codes_CodeId").IsUnique();

      entity.HasIndex(e => e.LastModifierId, "IX_Codes_LastModifierId");

      entity.HasIndex(e => e.ProjectId, "IX_Codes_ProjectId");

      entity.Property(e => e.CreatedDate).HasMaxLength(6);
      entity.Property(e => e.LastModifiedDate).HasMaxLength(6);

      entity.HasOne(d => d.Author).WithMany(p => p.CodeAuthors).HasForeignKey(d =>
d.AuthorId);

      entity.HasOne(d => d.LastModifier).WithMany(p =>
p.CodeLastModifiers).HasForeignKey(d => d.LastModifierId);

      entity.HasOne(d => d.Project).WithMany(p => p.Codes).HasForeignKey(d =>
d.ProjectId);
    });

    modelBuilder.Entity<CodeBasis>(entity =>
    {
      entity.HasKey(e => e.CodeBaseId).HasName("PRIMARY");

      entity.HasIndex(e => e.CodeBaseId, "IX_CodeBases_CodeBaseId").IsUnique();

      entity.HasIndex(e => e.CodeBaseId1, "IX_CodeBases_CodeBaseId1");

      entity.HasIndex(e => e.CodeId, "IX_CodeBases_CodeId");

      entity.HasIndex(e => e.ProjectId, "IX_CodeBases_ProjectId");

      entity.HasOne(d => d.CodeBaseId1Navigation).WithMany(p =>
p.InverseCodeBaseId1Navigation).HasForeignKey(d => d.CodeBaseId1);

      entity.HasOne(d => d.Code).WithMany(p => p.CodeBases).HasForeignKey(d =>
d.CodeId);

      entity.HasOne(d => d.Project).WithMany(p => p.CodeBases).HasForeignKey(d =>
d.ProjectId);
    });

    modelBuilder.Entity<Collab>(entity =>
    {
```

```
      entity.HasKey(e => e.CollabId).HasName("PRIMARY");

      entity.HasIndex(e => e.CollabId, "IX_Collabs_CollabId").IsUnique();

      entity.HasIndex(e => e.ProjectId, "IX_Collabs_ProjectId");

      entity.HasIndex(e => e.UserId, "IX_Collabs_UserId");

      entity.HasOne(d => d.Project).WithMany(p => p.Collabs).HasForeignKey(d =>
d.ProjectId);

      entity.HasOne(d => d.User).WithMany(p => p.Collabs).HasForeignKey(d =>
d.UserId);
    });

    modelBuilder.Entity<EfmigrationsHistory>(entity =>
    {
      entity.HasKey(e => e.MigrationId).HasName("PRIMARY");

      entity.ToTable("__EFMigrationsHistory");

      entity.Property(e => e.MigrationId).HasMaxLength(150);
      entity.Property(e => e.ProductVersion).HasMaxLength(32);
    });

    modelBuilder.Entity<Project>(entity =>
    {
      entity.HasKey(e => e.ProjectId).HasName("PRIMARY");

      entity.HasIndex(e => e.AdminId, "IX_Projects_AdminId");

      entity.HasIndex(e => e.ProjectId, "IX_Projects_ProjectId").IsUnique();

      entity.HasOne(d => d.Admin).WithMany(p => p.Projects).HasForeignKey(d =>
d.AdminId);
    });

    modelBuilder.Entity<User>(entity =>
    {
      entity.HasKey(e => e.UserId).HasName("PRIMARY");

      entity.HasIndex(e => e.UserId, "IX_Users_UserId").IsUnique();
    });

    OnModelCreatingPartial(modelBuilder);
  }

  partial void OnModelCreatingPartial(ModelBuilder modelBuilder);
}
```

Models

User

```
using System;
using System.Collections.Generic;

namespace CodingEnvironment_DF.Models;

public partial class User
{
  public int UserId { get; set; }

  public string? UserName { get; set; }

  public string? UserEmail { get; set; }

  public string? UserPassword { get; set; }

  public virtual ICollection<Code> CodeAuthors { get; set; } = new List<Code>();

  public virtual ICollection<Code> CodeLastModifiers { get; set; } = new
List<Code>();

  public virtual ICollection<Collab> Collabs { get; set; } = new List<Collab>();

  public virtual ICollection<Project> Projects { get; set; } = new List<Project>();
}
```

Project

```
using System;
using System.Collections.Generic;

namespace CodingEnvironment_DF.Models;

public partial class Project
{
  public int ProjectId { get; set; }

  public string? ProjectName { get; set; }

  public string? ProjectDescription { get; set; }

  public int AdminId { get; set; }

  public virtual User Admin { get; set; } = null!;

  public virtual ICollection<CodeBasis> CodeBases { get; set; } = new
List<CodeBasis>();

  public virtual ICollection<Code> Codes { get; set; } = new List<Code>();

  public virtual ICollection<Collab> Collabs { get; set; } = new List<Collab>();
}
```

Collab

```
using System;
using System.Collections.Generic;

namespace CodingEnvironment_DF.Models;

public partial class Collab
{
  public int CollabId { get; set; }

  public int UserId { get; set; }

  public int ProjectId { get; set; }

  public virtual Project Project { get; set; } = null!;

  public virtual User User { get; set; } = null!;
}
```

Code Base

```
using System;
using System.Collections.Generic;

namespace CodingEnvironment_DF.Models;

public partial class CodeBasis
{
  public int CodeBaseId { get; set; }

  public int ProjectId { get; set; }

  public int CodeId { get; set; }

  public int? CodeBaseId1 { get; set; }

  public virtual Code Code { get; set; } = null!;

  public virtual CodeBasis? CodeBaseId1Navigation { get; set; }

  public virtual ICollection<CodeBasis> InverseCodeBaseId1Navigation { get; set; }
= new List<CodeBasis>();

  public virtual Project Project { get; set; } = null!;
}
```

Code

```
using System;
using System.Collections.Generic;

namespace CodingEnvironment_DF.Models;

public partial class Code
{
  public int CodeId { get; set; }

  public string? CodeTitle { get; set; }

  public string? CodeText { get; set; }

  public DateTime CreatedDate { get; set; }

  public DateTime LastModifiedDate { get; set; }

  public int ProjectId { get; set; }

  public int AuthorId { get; set; }

  public int LastModifierId { get; set; }

  public virtual User Author { get; set; } = null!;

  public virtual ICollection<CodeBasis> CodeBases { get; set; } = new
List<CodeBasis>();

  public virtual User LastModifier { get; set; } = null!;

  public virtual Project Project { get; set; } = null!;
}
```

EfmigrationsHistory

```
using System;
using System.Collections.Generic;

namespace CodingEnvironment_DF.Models;

public partial class EfmigrationsHistory
{
  public string MigrationId { get; set; } = null!;

  public string ProductVersion { get; set; } = null!;
}
```

Repositories

User

```
namespace CodingEnvironment.Repo;

public class UserRepo
{
  public static void RegisterUser()
  {
    Console.Clear();
    Console.WriteLine("Adding a new User...");
    Console.WriteLine();

    var userName = GetUserName();
    Console.WriteLine();
    var userEmail = GetUserEmail();
    Console.WriteLine();
    var userPassword = GetUserPassword();
    Console.WriteLine();

    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var user = new User
        {
          UserName = userName,
          UserEmail = userEmail,
          UserPassword = userPassword
        };

        Console.WriteLine("Adding User to database...");
        context.Users.Add(user);
        context.SaveChanges();
        Console.WriteLine("User added successfully!\n");

        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine($"\nAn error occurred: {ex.Message}...\n");

      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }
```

```
  public static List<User> LoginUser()
  {
    Console.Clear();
    Console.WriteLine("Login to your account...");
    Console.WriteLine();

    var userEmail = GetUserEmail();
    Console.WriteLine();
    var userPassword = GetUserPassword();
    Console.WriteLine();

    try
    {
      using (var context = new CodeEnvDBContext()) {
        var users = context.Users.Where(u => u.UserEmail == userEmail &&
u.UserPassword == userPassword).ToList();
        if (users.Count > 0)
        {
          Console.WriteLine("Login successful!\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return users;
        }
        else
        {
          Console.WriteLine("Login failed. Please try again.\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return null;
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine($"\nAn error occurred: {ex.Message}...\n");
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
      return null;
    }
  }

  public static void UpdateUser(int UserIdToUpdate)
  {
    Console.Clear();
    Console.WriteLine("Updating User...");
    Console.WriteLine();

    var userName = GetUserName();
    Console.WriteLine();
    var userEmail = GetUserEmail();
```

```
  Console.WriteLine();
  var userPassword = GetUserPassword();
  Console.WriteLine();

  try
  {
    using (var context = new CodeEnvDBContext())
    {
      var user = context.Users.Find(UserIdToUpdate);
      user.UserName = userName;
      user.UserEmail = userEmail;
      user.UserPassword = userPassword;

      Console.WriteLine("Updating User in database...");
      context.SaveChanges();
      Console.WriteLine("User updated successfully!\n");

      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }
  catch (Exception ex)
  {
    Console.WriteLine($"\nAn error occurred: {ex.Message}...\n");
    Console.Write("Please press any key to continue...");
    Console.ReadKey();
  }
}

private static string GetUserName()
{
  while (true)
  {
    Console.Write("Enter User Name: ");
    var userName = Console.ReadLine();
    if (userName == null)
    {
      Console.WriteLine("User name cannot be empty.\n");
      continue;
    }
    else
    {
      return userName;
    }
  }
}

private static string GetUserEmail()
{
  while (true)
  {
```

```
        Console.Write("Enter User Email: ");
        var userEmail = Console.ReadLine();

        Regex regex = new Regex(@"^([\w\.\-]+)@([\w\-]+)((\.(\w){2,3})+)$");

        if (userEmail == null)
        {
          Console.WriteLine("User email cannot be empty.\n");
          continue;
        }
        else if (!regex.IsMatch(userEmail))
        {
          Console.WriteLine("User email is not valid.\n");
          continue;
        }
        else
        {
          return userEmail;
        }
      }
    }

    private static string GetUserPassword()
    {
      while (true)
      {
        Console.Write("Enter User Password: ");
        var userPassword = Console.ReadLine();
        if (userPassword == null)
        {
          Console.WriteLine("User password cannot be empty.\n");
          continue;
        }
        else
        {
          return userPassword;
        }
      }
    }
}
```

Project

```
using System.Reflection.Metadata.Ecma335;

namespace CodingEnvironment.Repo;

public class ProjectRepo
{
  public static void CreateProject(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Creating a new Project...");
    Console.WriteLine();

    var projectName = GetProjectName();
    Console.WriteLine();
    var projectDescription = GetProjectDescription();
    Console.WriteLine();
    var adminId = UserId;

    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var project = new Project
        {
          ProjectName = projectName,
          ProjectDescription = projectDescription,
          AdminId = adminId
        };

        Console.WriteLine("Adding Project to database...");
        context.Projects.Add(project);
        context.SaveChanges();
        Console.WriteLine("Project created successfully!\n");

        var collab = new Collab
        {
          UserId = UserId,
          ProjectId = project.ProjectId
        };

        Console.WriteLine("Adding Collab to database...");
        context.Collabs.Add(collab);
        context.SaveChanges();
        Console.WriteLine("Collab added successfully!\n");

        Console.Write("Please press any key to continue...");
```

```
        Console.ReadKey();
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }

  public static void ViewProjects(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Viewing Projects...");
    Console.WriteLine();

    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();

        if (collabs.Any())
        {
          foreach (var collab in collabs)
          {
            if (collab.UserId == UserId)
            {
              var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
              foreach (var project in projects)
              {
                Console.WriteLine($"- Project ID: {project.ProjectId}: Project
Name: {project.ProjectName} Description: {project.ProjectDescription}");
              }
            }
          }

          Console.WriteLine();
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
        }
        else
        {
          Console.WriteLine("You are not a member of any project. Please create a
project first.");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return;
```

```
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }

  public static void UpdateProject(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Updating Project...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();

      if (collabs.Any())
      {
        foreach (var collab in collabs)
        {
          if (collab.UserId == UserId)
          {
            var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
            foreach (var project in projects)
            {
              Console.WriteLine($"- Project ID: {project.ProjectId}: Project Name:
{project.ProjectName} Description: {project.ProjectDescription}");
            }
          }
        }
      }
      else
      {
        Console.WriteLine("You are not a member of any project. Please create a
project first.");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
        return;
      }
    }

    Console.WriteLine();
    var projectId = GetProjectId();
    Console.WriteLine();
```

```
    try
    {
      using (var context = new CodeEnvDBContext())
      {
        var project = context.Projects.Find(projectId);
        if (project == null)
        {
          Console.WriteLine("Project not found.\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return;
        }
        else
        {
          Console.WriteLine("Updating Project in database...");
          var projectName = GetProjectName();
          Console.WriteLine();
          var projectDescription = GetProjectDescription();
          Console.WriteLine();

          project.ProjectName = projectName;
          project.ProjectDescription = projectDescription;

          context.SaveChanges();
          Console.WriteLine("Project updated successfully!\n");

          Console.Write("Please press any key to continue...");
          Console.ReadKey();
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }

  public static void DeleteProject(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Deleting Project...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();
```

```
      if (collabs.Any())
      {
        foreach (var collab in collabs)
        {
          if (collab.UserId == UserId)
          {
            var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
            foreach (var project in projects)
            {
              Console.WriteLine($"- Project ID: {project.ProjectId}: Project Name:
{project.ProjectName} Description: {project.ProjectDescription}");
            }
          }
        }
      }
      else
      {
        Console.WriteLine("You are not a member of any project. Please create a
project first.");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
        return;
      }
    }

    Console.WriteLine();
    Console.WriteLine("Please enter the ID of the project you want to delete: ");
    var projectId = GetProjectId();

    try
    {
      Console.WriteLine("Deleting Project from database..." + projectId);
      using (var context = new CodeEnvDBContext())
      {
        var project = context.Projects.Find(projectId);
        if (project == null)
        {
          Console.WriteLine("Project not found.\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
          return;
        }
        else
        {
          context.Projects.Remove(project);
          context.SaveChanges();
          var collabs = context.Collabs.Where(c => c.ProjectId ==
projectId).ToList();
          foreach (var collab in collabs)
          {
```

```
            context.Collabs.Remove(collab);
            context.SaveChanges();
          }
          Console.WriteLine("Project deleted successfully!\n");
          Console.Write("Please press any key to continue...");
          Console.ReadKey();
        }
      }
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
}


private static int? GetProjectId()
{
  while (true)
  {
    Console.WriteLine("Enter Project ID: ");
    int? projectId = int.Parse(Console.ReadLine());
    if (projectId == null)
    {
      Console.WriteLine("Project ID cannot be empty.\n");
      continue;
    }
    else
    {
      return projectId;
    }
  }
}

private static string GetProjectName()
{
  while (true)
  {
    Console.WriteLine("Enter Project Name: ");
    var projectName = Console.ReadLine();
    if (projectName == null)
    {
      Console.WriteLine("Project name cannot be empty.\n");
      continue;
    }
    else
    {
      return projectName;
    }
```

```
    }
  }

  private static string GetProjectDescription()
  {
    while (true)
    {
      Console.WriteLine("Enter Project Description: ");
      var projectDescription = Console.ReadLine();
      if (projectDescription == null)
      {
        Console.WriteLine("Project description cannot be empty.\n");
        continue;
      }
      else
      {
        return projectDescription;
      }
    }
  }
}
```

Code

```
namespace CodingEnvironment.Repo;

public class CodeRepo
{
  public static int? SelectProj(int UserId)
  {
    Console.Clear();
    Console.WriteLine("Selecting Project...");
    Console.WriteLine();
    using (var context = new CodeEnvDBContext())
    {
      var collabs = context.Collabs.ToList().Where(c => c.UserId ==
UserId).ToList();

      if (collabs.Any())
      {
        foreach (var collab in collabs)
        {
          if (collab.UserId == UserId)
          {
            var projects = context.Projects.Where(p => p.ProjectId ==
collab.ProjectId).ToList();
            foreach (var project in projects)
            {
              Console.WriteLine($"- Project ID: {project.ProjectId}: Project Name:
{project.ProjectName} Description: {project.ProjectDescription}");
            }
          }
        }
      }
      else
      {
        Console.WriteLine("You are not a member of any project. Please create a
project first.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
        return null;
      }
    }

    Console.WriteLine();
    Console.WriteLine("Select Project ID: ");
    int? projectId = int.Parse(Console.ReadLine());
    return projectId;
  }
```

```
public static void AddCode(int UserId)
{
  int? selectedProjectId = SelectProj(UserId);

  if (selectedProjectId == null)
  {
    return;
  }

  Console.Clear();
  Console.WriteLine("Adding Code...");
  Console.WriteLine();

  var codeTitle = GetCodeTitle();
  Console.WriteLine();
  var codeText = GetCodeText();
  Console.WriteLine();

  try
  {
    using (var context = new CodeEnvDBContext())
    {
      var code = new Code
      {
        CodeTitle = codeTitle,
        CodeText = codeText,
        ProjectId = selectedProjectId.Value,
        CreatedDate = DateTime.Now,
        LastModifiedDate = DateTime.Now,
        AuthorId = UserId,
        LastModifierId = UserId,
      };

      Console.WriteLine("Adding Code to database...");
      context.Codes.Add(code);
      context.SaveChanges();
      Console.WriteLine("Code added successfully!\n");

      var codebase = new CodeBasis
      {
        ProjectId = selectedProjectId.Value,
        CodeId = code.CodeId
      };
      Console.WriteLine("Adding Codebase to database...");
      context.CodeBases.Add(codebase);
      context.SaveChanges();
      Console.WriteLine("Codebase added successfully!\n");

      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
```

```
    }
    catch (Exception ex)
    {
      Console.WriteLine("An error occurred: " + ex.Message);
      Console.Write("Please press any key to continue...");
      Console.ReadKey();
    }
  }



  public static void ViewCode(int UserId)
  {
    int? selectedProjectId = SelectProj(UserId);

    if (selectedProjectId == null)
    {
      return;
    }

    Console.Clear();
    Console.WriteLine("Viewing Code...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var codebase = context.CodeBases.Where(c => c.ProjectId ==
selectedProjectId).ToList();
      if (codebase.Any())
      {
        foreach (var codeBase in codebase)
        {
          var codeView = context.Codes.Where(c => c.CodeId ==
codeBase.CodeId).FirstOrDefault();
          Console.WriteLine($"- Code ID: {codeView.CodeId}: Code Title:
{codeView.CodeTitle}");
        }

        Console.WriteLine();
        Console.Write("Enter the Code ID you want to view: ");
        var codeId = int.Parse(Console.ReadLine());

        var code = context.Codes.Where(c => c.CodeId == codeId).FirstOrDefault();
        Console.WriteLine($"- Code Title: {code.CodeTitle}");
        Console.WriteLine($"- Code Text: \n{code.CodeText}");

        Console.WriteLine();
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
      else
```

```
      {
        Console.WriteLine("You have no codes to view.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
  }

  public static void UpdateCode(int UserId)
  {
    int? selectedProjectId = SelectProj(UserId);

    if (selectedProjectId == null)
    {
      return;
    }

    Console.Clear();
    Console.WriteLine("Updating Code...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var codebase = context.CodeBases.Where(c => c.ProjectId ==
selectedProjectId).ToList();
      if (codebase.Any())
      {
        foreach (var codeBase in codebase)
        {
          var codeView = context.Codes.Where(c => c.CodeId ==
codeBase.CodeId).FirstOrDefault();
          Console.WriteLine($"- Code ID: {codeView.CodeId}: Code Title:
{codeView.CodeTitle}");
        }

        Console.WriteLine();
        Console.Write("Enter the Code ID you want to view: ");
        var codeId = int.Parse(Console.ReadLine());
        foreach (var code in codebase)
        {
          if (code.CodeId == codeId)
          {
            Console.WriteLine("\nUpdating Code...");
            Console.WriteLine();
            var newCodeTitle = GetCodeTitle();
            Console.WriteLine();
            var newCodeText = GetCodeText();
            code.Code.CodeTitle = newCodeTitle;
            code.Code.CodeText = newCodeText;
            context.SaveChanges();
            Console.WriteLine("Code updated successfully!\n");
```

```
            break;
          }
          else
          {
            Console.WriteLine("Invalid Code ID. Please try again.");
          }
        }

        Console.WriteLine();
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }else
      {
        Console.WriteLine("You have no codes to update.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
  }

  public static void DeleteCode(int UserId)
  {
    int? selectedProjectId = SelectProj(UserId);

    if (selectedProjectId == null)
    {
      return;
    }

    Console.Clear();
    Console.WriteLine("Deleting Code...");
    Console.WriteLine();

    using (var context = new CodeEnvDBContext())
    {
      var codebase = context.CodeBases.Where(c => c.ProjectId ==
selectedProjectId).ToList();
      if (codebase.Any())
      {
        foreach (var codeBase in codebase)
        {
          var codeView = context.Codes.Where(c => c.CodeId ==
codeBase.CodeId).FirstOrDefault();
          Console.WriteLine($"- Code ID: {codeView.CodeId}: Code Title:
{codeView.CodeTitle}");
        }

        Console.WriteLine();
        Console.Write("Enter the Code ID you want to delete: ");
        var codeId = int.Parse(Console.ReadLine());
        foreach (var code in codebase)
```

```
        {
          if (code.CodeId == codeId)
          {
            Console.WriteLine("\nDeleting Code...");
            context.Codes.Remove(code.Code);

            var codebaseToDelete = context.CodeBases.Where(c => c.CodeId ==
codeId).FirstOrDefault();
            context.CodeBases.Remove(codebaseToDelete);
            context.SaveChanges();
            Console.WriteLine("Code deleted successfully!\n");
          }
          else
          {
            Console.WriteLine("Invalid Code ID. Please try again.");
          }
        }

        Console.WriteLine();
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
      else
      {
        Console.WriteLine("You have no codes to delete.\n");
        Console.Write("Please press any key to continue...");
        Console.ReadKey();
      }
    }
  }

  private static string GetCodeTitle()
  {
    while (true)
    {
      Console.Write("Enter Code Title: ");
      var codeTitle = Console.ReadLine();
      if (codeTitle == null)
      {
        Console.WriteLine("Code title cannot be empty.\n");
        continue;
      }
      else
      {
        return codeTitle;
      }
    }
  }

  private static string GetCodeText()
  {
```

```
    while (true)
    {
      Console.WriteLine("Enter Code Text: ");
      var codeText = Console.ReadLine();
      if (codeText == null)
      {
        Console.WriteLine("Code text cannot be empty.\n");
        continue;
      }
      else
      {
        return codeText;
      }
    }
  }


}
```

Global Using

```
global using Microsoft.EntityFrameworkCore;

global using System.Text.RegularExpressions;

global using System.ComponentModel.DataAnnotations;
global using System.ComponentModel.DataAnnotations.Schema;

global using CodingEnvironment_DF.Models;
global using CodingEnvironment.Repo;
```

Program.cs

```
public class Program
{
  public static void Main(string[] args)
  {
    while (true)
    {
      Console.Clear();
      Console.WriteLine("Welcome to Collabo - Your Coding Environment...");

      Console.WriteLine("\nSelect an option:");
      Console.WriteLine();
      Console.WriteLine("1. Login \n2. Register \n0. Exit");

      Console.Write("\nEnter your choice: ");
      var choice = Console.ReadLine();

      switch (choice)
      {
        case "1":
          {
            List<User> LoggedIn = UserRepo.LoginUser();
            if (LoggedIn.Any())
            {
              Logged(LoggedIn);
            }
            else
            {
              Console.WriteLine("Login failed. Please try again.\n");
              Console.Write("Press any key to continue...");
              Console.ReadKey();
              continue;
            }
            break;
          }
        case "2":
          {
            UserRepo.RegisterUser();
            break;
          }
        case "0":
          {
            Environment.Exit(0);
            break;
          }
        default:
          {
            Console.WriteLine("Invalid choice. Please try again.");
            break;
```

```
        }
    }
  }
}

public static void Logged(List<User> users)
{
  while (true)
  {
    Console.Clear();
    Console.WriteLine("Welcome " + users[0].UserName + "!");
    Console.WriteLine();

    Console.WriteLine("Select an option:");
    Console.WriteLine();
    Console.WriteLine("User: 1. Update User Details.");
    Console.WriteLine("Project: 2. Create 3. View 4. Update  5. Delete");
    Console.WriteLine("Code: 6. Add 7. View 8. Update 9. Delete");
    Console.WriteLine($"Logout: 0. Logout as {users[0].UserName}");

    Console.Write("\nEnter your choice: ");
    var choice = Console.ReadLine();

    switch (choice)
    {
      case "1":
      {
        UserRepo.UpdateUser(users[0].UserId);
        break;
      }
      case "2":
      {
        ProjectRepo.CreateProject(users[0].UserId);
        break;
      }
      case "3":
      {
        ProjectRepo.ViewProjects(users[0].UserId);
        break;
      }
      case "4":
      {
        ProjectRepo.UpdateProject(users[0].UserId);
        break;
      }
      case "5":
        {
          ProjectRepo.DeleteProject(users[0].UserId);
          break;
        }
        case "6":
```

```
            {
              CodeRepo.AddCode(users[0].UserId);
              break;
            }
            case "7":
            {
              CodeRepo.ViewCode(users[0].UserId);
              break;
            }
            case "8":
            {
              CodeRepo.UpdateCode(users[0].UserId);
              break;
            }
            case "9":
            {
              CodeRepo.DeleteCode(users[0].UserId);
              break;
            }
          case "0":
          {
            return;
          }
          default:
            {
              Console.WriteLine("Invalid choice. Please try again.");
              break;
            }
        }
      }
    }
}
```